Frontend Development Plan

Startup Investment Readiness & Deal Intelligence Platform

Executive Summary

This frontend plan delivers a **dual-user experience**: a powerful internal tool for **VCs** to evaluate deals and a guided journey for **founders** to improve fundraising readiness. The UI must be data-dense yet intuitive, professional yet approachable, and optimized for rapid decision-making.

---

1. Product Strategy & User Personas

Primary User Personas

Persona 1: VC Analyst (Sarah, 26)

Goals: Screen 50+ decks/week, identify high-potential deals fast
Pain Points: Information overload, subjective criteria, manual data entry
Needs: Dashboard with clear signals, quick comparisons, exportable reports

Persona 2: VC Partner (Rajesh, 42)

Goals: Make IC-ready recommendations, portfolio analysis
Pain Points: Lack of standardized evaluation, time constraints
Needs: High-level insights, trend analysis, deal flow metrics

Persona 3: Founder (Priya, 31)

Goals: Understand why she's not fundraise-ready, improve pitch
Pain Points: Vague rejection reasons, unrealistic valuation expectations
Needs: Actionable feedback, benchmarking data, investor discovery

Persona 4: VC Admin/Operations (Amit, 35)

Goals: Manage platform, track usage, ensure compliance
Pain Points: Manual user management, no visibility into platform ROI
Needs: Admin dashboard, analytics, user management tools

---

2. Application Architecture

2.1 Technology Stack

```
// Core Framework
- React 18+ (with TypeScript)
- Next.js 14 (App Router)
- TailwindCSS + shadcn/ui

// State Management
- Zustand (global state)
- React Query / TanStack Query (server state)
- React Hook Form (forms)

// Data Visualization
- Recharts (charts)
- D3.js (custom visualizations)
- Lucide React (icons)

// File Handling
- react-dropzone (file uploads)
- PDF.js (preview)
- mammoth.js (DOCX preview)

// Real-time
- Socket.io-client (WebSocket)
- React Query subscriptions

// Auth
- NextAuth.js (OAuth integration)
- JWT token management

// Developer Tools
- ESLint + Prettier
- TypeScript strict mode
- Storybook (component library)
- Vitest + React Testing Library
```

2.2 Folder Structure

```
frontend/
├── app/                    # Next.js App Router
│   ├── (auth)/             # Auth routes
│   │   ├── login/
│   │   ├── register/
│   │   └── forgot-password/
│   ├── (dashboard)/        # Main app
```

```
│   │   ├── layout.tsx
│   │   ├── vc/              # VC-specific routes
│   │   │   ├── deal-flow/
│   │   │   ├── analytics/
│   │   │   ├── startups/[id]/
│   │   │   └── settings/
│   │   └── founder/         # Founder-specific routes
│   │       ├── dashboard/
│   │       ├── pitch-analyzer/
│   │       ├── readiness/
│   │       ├── valuation/
│   │       ├── investor-match/
│   │       ├── communities/
│   │       └── crm/
│   └── api/                 # API routes (proxy)
├── components/
│   ├── ui/                  # shadcn/ui components
│   ├── shared/              # Shared components
│   │   ├── Navigation/
│   │   ├── FileUpload/
│   │   ├── DataTable/
│   │   └── Charts/
│   ├── vc/                  # VC-specific components
│   │   ├── DealFlowTable/
│   │   ├── StartupCard/
│   │   └── AnalyticsWidget/
│   └── founder/             # Founder-specific components
│       ├── ReadinessScore/
│       ├── PitchAnalyzer/
│       ├── InvestorCard/
│       └── TimelineTracker/
├── lib/
│   ├── api/                 # API client
│   ├── hooks/               # Custom hooks
│   ├── utils/               # Utilities
│   └── stores/              # Zustand stores
├── styles/
│   └── globals.css
├── public/
│   └── assets/
└── types/
    └── index.ts
```

---

3. Design System

3.1 Color Palette

```
// Tailwind config extension
const colors = {
  brand: {
    primary: '#2563EB',      // Blue (trust, professional)
    secondary: '#7C3AED',    // Purple (innovation)
    success: '#10B981',      // Green (positive signals)
    warning: '#F59E0B',      // Amber (caution)
    danger: '#EF4444',       // Red (risk)
  },
  neutral: {
    50: '#F9FAFB',
    100: '#F3F4F6',
    200: '#E5E7EB',
    300: '#D1D5DB',
    400: '#9CA3AF',
    500: '#6B7280',
    600: '#4B5563',
    700: '#374151',
    800: '#1F2937',
    900: '#111827',
  },
  // Scoring colors
  score: {
    excellent: '#10B981',    // 80-100
    good: '#3B82F6',         // 60-79
    average: '#F59E0B',      // 40-59
    poor: '#EF4444',         // 0-39
  }
}
```

3.2 Typography

```
// Font families
const fonts = {
  sans: ['Inter', 'sans-serif'],
  mono: ['JetBrains Mono', 'monospace'],
}

// Type scale
const typography = {
  h1: 'text-4xl font-bold',
  h2: 'text-3xl font-semibold',
  h3: 'text-2xl font-semibold',
  h4: 'text-xl font-semibold',
  body: 'text-base',
  small: 'text-sm',
  tiny: 'text-xs',
}
```

3.3 Component Library (shadcn/ui)

```
# Core components to install
npx shadcn-ui@latest add button
npx shadcn-ui@latest add card
npx shadcn-ui@latest add input
npx shadcn-ui@latest add select
npx shadcn-ui@latest add table
npx shadcn-ui@latest add tabs
npx shadcn-ui@latest add dialog
npx shadcn-ui@latest add dropdown-menu
npx shadcn-ui@latest add toast
npx shadcn-ui@latest add badge
npx shadcn-ui@latest add progress
npx shadcn-ui@latest add avatar
npx shadcn-ui@latest add separator
npx shadcn-ui@latest add skeleton
```

---

4. Key User Flows

4.1 Founder Journey: Pitch Deck Analysis

```
1. Upload Deck → 2. Processing State → 3. Analysis Results → 4. Action Items
```

Screen 1: Upload Deck

```
// components/founder/PitchAnalyzer/UploadScreen.tsx
import { FileUpload } from '@/components/shared/FileUpload'

export function UploadScreen() {
  return (
    <div className="max-w-3xl mx-auto py-12">
      <h1>Analyze Your Pitch Deck</h1>
      <p className="text-neutral-600">
        Get AI-powered feedback in under 3 minutes
      </p>

      <FileUpload
        accept=".pdf,.ppt,.pptx"
        maxSize={50 * 1024 * 1024} // 50MB
        onUpload={handleUpload}
      />

      <form>
        <Select label="Sector" options={sectors} />
        <Select label="Stage" options={stages} />
```

```
        <Input label="Monthly Revenue (Optional)" type="number" />
        <Input label="Team Size" type="number" />
      </form>

      <Button onClick={analyzeNow}>Analyze Now</Button>
    </div>
  )
}
```

Screen 2: Processing State

```
// components/founder/PitchAnalyzer/ProcessingScreen.tsx
export function ProcessingScreen() {
  const [progress, setProgress] = useState(0)

  return (
    <div className="flex flex-col items-center justify-center h-screen">
      <Loader className="w-16 h-16 animate-spin" />
      <h2>Analyzing your pitch deck...</h2>
      <Progress value={progress} max={100} />

      <div className="mt-8 text-sm text-neutral-500">
        <p>✓ Extracting content from slides</p>
        <p>⏳ Evaluating market opportunity</p>
        <p className="opacity-50">⏳ Analyzing team strength</p>
      </div>

      <p className="mt-4 text-xs">
        Estimated time: 2-3 minutes
      </p>
    </div>
  )
}
```

Screen 3: Analysis Results

```
// components/founder/PitchAnalyzer/ResultsScreen.tsx
export function ResultsScreen({ analysis }) {
  return (
    <div className="max-w-6xl mx-auto py-8">
      {/* Hero Score */}
      <Card className="mb-8">
        <div className="text-center py-12">
          <ScoreRing score={analysis.overall_score} size="xl" />
          <h1 className="text-5xl font-bold mt-4">
            {analysis.overall_score}/100
          </h1>
          <p className="text-xl text-neutral-600">
```

```
      Overall Pitch Score
    </p>
    <Badge variant={getScoreVariant(analysis.overall_score)}>
      {getScoreLabel(analysis.overall_score)}
    </Badge>
  </div>
</Card>

{/* Component Scores */}
<div className="grid grid-cols-3 gap-4 mb-8">
  <ScoreCard
    title="Idea Novelty"
    score={analysis.scores.idea}
    icon={<Lightbulb />}
  />
  <ScoreCard
    title="Team Capability"
    score={analysis.scores.team}
    icon={<Users />}
  />
  <ScoreCard
    title="Market Opportunity"
    score={analysis.scores.market}
    icon={<TrendingUp />}
  />
  <ScoreCard
    title="Traction"
    score={analysis.scores.traction}
    icon={<Target />}
  />
  <ScoreCard
    title="Risk Management"
    score={analysis.scores.risk}
    icon={<Shield />}
  />
  <ScoreCard
    title="Scalability"
    score={analysis.scores.scalability}
    icon={<Rocket />}
  />
</div>

{/* Flags */}
<Tabs defaultValue="red">
  <TabsList>
    <TabsTrigger value="red">
      🚩 Red Flags ({analysis.flags.red.length})
    </TabsTrigger>
    <TabsTrigger value="amber">
      ⚠️ Amber Flags ({analysis.flags.amber.length})
```

```
    </TabsTrigger>
    <TabsTrigger value="green">
      ✅ Green Flags ({analysis.flags.green.length})
    </TabsTrigger>
  </TabsList>

  <TabsContent value="red">
    <FlagsList flags={analysis.flags.red} type="danger" />
  </TabsContent>
  {/* ... */}
</Tabs>

{/* Actionable Suggestions */}
<Card className="mt-8">
  <h2>How to Improve Your Pitch</h2>
  <ol className="space-y-4">
    {analysis.suggestions.map((suggestion, i) => (
      <li key={i}>
        <div className="flex items-start gap-3">
          <Badge>{i + 1}</Badge>
          <div>
            <h4 className="font-semibold">{suggestion.title}</h4>
            <p className="text-neutral-600">{suggestion.description}</p>
          </div>
        </div>
      </li>
    ))}
  </ol>
</Card>

{/* Comparative Insights */}
<Card className="mt-8">
  <h3>How You Compare</h3>
  <p className="text-neutral-600">
    vs. other {analysis.sector} startups at {analysis.stage} stage
  </p>
  <PercentileChart percentile={analysis.percentile_rank} />
  <p className="text-sm text-neutral-500 mt-4">
    You scored better than {analysis.percentile_rank}% of similar pitches
  </p>
</Card>

{/* Actions */}
<div className="flex gap-4 mt-8">
  <Button onClick={downloadReport}>
    <Download className="mr-2" /> Download Report
  </Button>
  <Button variant="outline" onClick={shareWithTeam}>
    <Share className="mr-2" /> Share with Team
  </Button>
```

```
        <Button variant="outline" onClick={checkReadiness}>
          Check Investment Readiness →
        </Button>
      </div>
    </div>
  )
}
```

4.2 Founder Journey: Investment Readiness

```
// app/founder/readiness/page.tsx
export default function ReadinessPage() {
  const { data: readiness, isLoading } = useQuery({
    queryKey: ['readiness'],
    queryFn: () => api.getReadinessScore(startupId)
  })

  if (isLoading) return <ReadinessSkeleton />

  return (
    <div className="max-w-6xl mx-auto py-8">
      {/* Overall Readiness */}
      <Card className="mb-8 bg-gradient-to-br from-blue-500 to-purple-600 text-white">
        <div className="p-8">
          <h1 className="text-3xl font-bold mb-2">
            Investment Readiness Score
          </h1>
          <div className="flex items-end gap-4">
            <span className="text-7xl font-bold">
              {readiness.overall_score}
            </span>
            <span className="text-4xl mb-2">/100</span>
          </div>
          <Badge variant="secondary" className="mt-4">
            {readiness.readiness_level}
          </Badge>

          {readiness.readiness_level === 'READY' && (
            <div className="mt-4 flex items-center gap-2">
              <CheckCircle className="w-6 h-6" />
              <span className="text-lg">
                You're ready to start fundraising!
              </span>
            </div>
          )}

          {readiness.readiness_level === 'NEARLY_READY' && (
            <div className="mt-4">
              <p className="text-lg mb-2">
```

```
          Estimated time to readiness:
        </p>
        <p className="text-3xl font-bold">
          {readiness.time_to_readiness_days} days
        </p>
      </div>
    )}
  </div>
</Card>

{/* Score Breakdown */}
<Card className="mb-8">
  <h2>Score Breakdown</h2>
  <div className="space-y-4">
    <ReadinessComponent
      label="Pitch Deck Quality"
      score={readiness.pitch_quality_score}
      weight={30}
      maxScore={100}
    />
    <ReadinessComponent
      label="Traction Metrics"
      score={readiness.traction_score}
      weight={25}
      maxScore={100}
    />
    <ReadinessComponent
      label="Team Strength"
      score={readiness.team_score}
      weight={20}
      maxScore={100}
    />
    <ReadinessComponent
      label="Market Timing"
      score={readiness.market_timing_score}
      weight={15}
      maxScore={100}
    />
    <ReadinessComponent
      label="Financial Health"
      score={readiness.financial_health_score}
      weight={10}
      maxScore={100}
    />
  </div>
</Card>

{/* Improvement Roadmap */}
<Card className="mb-8">
  <h2>What You Need to Improve</h2>
```

```
        <div className="space-y-6">
         {readiness.improvements_needed.map((improvement, i) => (
           <ImprovementCard
             key={i}
             priority={improvement.priority}
             category={improvement.category}
             issue={improvement.issue}
             action={improvement.action}
             estimatedImpact={improvement.estimated_impact}
           />
         ))}
        </div>
      </Card>

      {/* Progress Tracking */}
      <Card>
        <h2>Your Progress Over Time</h2>
        <ReadinessChart data={readiness.historical_scores} />
      </Card>
    </div>
  )
}

// Component: ReadinessComponent
function ReadinessComponent({ label, score, weight, maxScore }) {
  const percentage = (score / maxScore) * 100
  const weightedScore = (score / maxScore) * weight

  return (
    <div>
      <div className="flex justify-between items-center mb-2">
        <span className="font-medium">{label}</span>
        <div className="flex items-center gap-2">
          <span className="text-2xl font-bold">{score}</span>
          <span className="text-neutral-500">/ {maxScore}</span>
          <Badge variant="outline">{weight}% weight</Badge>
        </div>
      </div>
      <Progress value={percentage} className="h-3" />
      <p className="text-sm text-neutral-500 mt-1">
        Contributes {weightedScore.toFixed(1)} points to overall score
      </p>
    </div>
  )
}
```

4.3 Founder Journey: Investor Matching

```
// app/founder/investor-match/page.tsx
export default function InvestorMatchPage() {
  const [filters, setFilters] = useState({
    sector: [],
    stage: [],
    geography: [],
    ticketSize: [0, 10] // Cr
  })

  const { data: matches } = useQuery({
    queryKey: ['investor-matches', filters],
    queryFn: () => api.getInvestorMatches(startupId, filters)
  })

  return (
    <div className="max-w-7xl mx-auto py-8">
      <div className="mb-8">
        <h1 className="text-3xl font-bold">Find Your Investors</h1>
        <p className="text-neutral-600">
          {matches?.length} investors matched based on your profile
        </p>
      </div>

      {/* Filters */}
      <Card className="mb-8">
        <div className="grid grid-cols-4 gap-4">
          <MultiSelect
            label="Sectors"
            options={SECTORS}
            value={filters.sector}
            onChange={(v) => setFilters({...filters, sector: v})}
          />
          <MultiSelect
            label="Stages"
            options={STAGES}
            value={filters.stage}
            onChange={(v) => setFilters({...filters, stage: v})}
          />
          <MultiSelect
            label="Geography"
            options={GEOGRAPHIES}
            value={filters.geography}
            onChange={(v) => setFilters({...filters, geography: v})}
          />
          <div>
            <label>Ticket Size (Cr)</label>
            <RangeSlider
              min={0}
              max={50}
```

```
            value={filters.ticketSize}
            onChange={(v) => setFilters({...filters, ticketSize: v})}
          />
        </div>
      </div>
    </Card>

    {/* Match Results */}
    <div className="space-y-4">
      {matches?.map((match) => (
        <InvestorMatchCard
          key={match.investor_id}
          investor={match.investor}
          matchScore={match.match_score}
          matchReasons={match.match_reasoning}
          onContact={() => logContactAttempt(match.investor_id)}
        />
      ))}
    </div>
  </div>
)
}

// Component: InvestorMatchCard
function InvestorMatchCard({ investor, matchScore, matchReasons, onContact }) {
  return (
    <Card className="hover:shadow-lg transition-shadow">
      <div className="flex items-start gap-6 p-6">
        {/* Match Score Badge */}
        <div className="flex-shrink-0">
          <div className="w-20 h-20 rounded-full bg-gradient-to-br from-blue-500 to-purple-600 flex items-center justify-ce
            <div className="text-center text-white">
              <div className="text-2xl font-bold">{matchScore}</div>
              <div className="text-xs">Match</div>
            </div>
          </div>
        </div>

        {/* Investor Info */}
        <div className="flex-grow">
          <div className="flex items-start justify-between">
            <div>
              <h3 className="text-xl font-semibold">{investor.name}</h3>
              <Badge>{investor.type}</Badge>
            </div>
            <Button onClick={onContact}>
              <Mail className="mr-2" /> Contact
            </Button>
          </div>
```

```
<div className="mt-4 grid grid-cols-3 gap-4">
  <div>
    <p className="text-sm text-neutral-500">Focus Sectors</p>
    <div className="flex flex-wrap gap-1 mt-1">
      {investor.sectors.map(s => (
        <Badge key={s} variant="outline">{s}</Badge>
      ))}
    </div>
  </div>
  <div>
    <p className="text-sm text-neutral-500">Stages</p>
    <div className="flex flex-wrap gap-1 mt-1">
      {investor.stages.map(s => (
        <Badge key={s} variant="outline">{s}</Badge>
      ))}
    </div>
  </div>
  <div>
    <p className="text-sm text-neutral-500">Ticket Size</p>
    <p className="font-medium">
      ₹{investor.ticket_size_min}Cr - ₹{investor.ticket_size_max}Cr
    </p>
  </div>
</div>

{/* Match Reasoning */}
<div className="mt-4">
  <p className="text-sm font-medium mb-2">Why this is a good match:</p>
  <ul className="space-y-1">
    {matchReasons.map((reason, i) => (
      <li key={i} className="flex items-start gap-2 text-sm text-neutral-600">
        <CheckCircle className="w-4 h-4 text-green-500 mt-0.5" />
        <span>{reason}</span>
      </li>
    ))}
  </ul>
</div>

{/* Portfolio Companies */}
<Collapsible className="mt-4">
  <CollapsibleTrigger className="text-sm text-blue-600 hover:underline">
    View portfolio companies ({investor.portfolio_companies.length})
  </CollapsibleTrigger>
  <CollapsibleContent>
    <div className="grid grid-cols-4 gap-2 mt-2">
      {investor.portfolio_companies.slice(0, 8).map(company => (
        <div key={company} className="text-xs bg-neutral-100 rounded px-2 py-1">
          {company}
        </div>
      ))}
```

```
            </div>
          </CollapsibleContent>
        </Collapsible>
      </div>
    </div>
  </Card>
)
}
```

4.4 VC Journey: Deal Flow Dashboard

```
// app/vc/deal-flow/page.tsx
export default function DealFlowPage() {
  const [view, setView] = useState<'table' | 'kanban'>('table')
  const [filters, setFilters] = useState({
    stage: [],
    sector: [],
    scoreRange: [0, 100],
    status: 'all'
  })

  const { data: deals } = useQuery({
    queryKey: ['deal-flow', filters],
    queryFn: () => api.getDealFlow(filters)
  })

  return (
    <div className="py-8">
      {/* Header */}
      <div className="flex items-center justify-between mb-8">
        <div>
          <h1 className="text-3xl font-bold">Deal Flow</h1>
          <p className="text-neutral-600">
            {deals?.length} active deals in pipeline
          </p>
        </div>

        <div className="flex items-center gap-4">
          <ToggleGroup value={view} onValueChange={setView}>
            <ToggleGroupItem value="table">
              <List className="mr-2" /> Table
            </ToggleGroupItem>
            <ToggleGroupItem value="kanban">
              <LayoutGrid className="mr-2" /> Kanban
            </ToggleGroupItem>
          </ToggleGroup>

          <Button onClick={exportToCsv}>
            <Download className="mr-2" /> Export
```

```
          </Button>
        </div>
      </div>

      {/* Filters */}
      <DealFlowFilters filters={filters} onChange={setFilters} />

      {/* View */}
      {view === 'table' ? (
        <DealFlowTable deals={deals} />
      ) : (
        <DealFlowKanban deals={deals} />
      )}
    </div>
  )
}

// Component: DealFlowTable
function DealFlowTable({ deals }) {
  const columns = [
    {
      accessorKey: 'startup_name',
      header: 'Startup',
      cell: ({ row }) => (
        <div className="flex items-center gap-3">
          <Avatar>
            <AvatarImage src={row.original.logo} />
            <AvatarFallback>{row.original.startup_name[0]}</AvatarFallback>
          </Avatar>
          <div>
            <p className="font-medium">{row.original.startup_name}</p>
            <p className="text-xs text-neutral-500">{row.original.tagline}</p>
          </div>
        </div>
      )
    },
    {
      accessorKey: 'sector',
      header: 'Sector',
      cell: ({ row }) => (
        <Badge variant="outline">{row.original.sector}</Badge>
      )
    },
    {
      accessorKey: 'stage',
      header: 'Stage',
    },
    {
      accessorKey: 'overall_score',
      header: 'Score',
```

```
    cell: ({ row }) => (
      <div className="flex items-center gap-2">
        <ScoreBadge score={row.original.overall_score} />
        <span className="text-lg font-semibold">
          {row.original.overall_score}
        </span>
      </div>
    ),
    sortingFn: 'basic'
  },
  {
    accessorKey: 'readiness_level',
    header: 'Readiness',
    cell: ({ row }) => (
      <ReadinessBadge level={row.original.readiness_level} />
    )
  },
  {
    accessorKey: 'submitted_at',
    header: 'Submitted',
    cell: ({ row }) => formatDistanceToNow(row.original.submitted_at)
  },
  {
    id: 'actions',
    cell: ({ row }) => (
      <DropdownMenu>
        <DropdownMenuTrigger asChild>
          <Button variant="ghost" size="sm">
            <MoreVertical className="w-4 h-4" />
          </Button>
        </DropdownMenuTrigger>
        <DropdownMenuContent align="end">
          <DropdownMenuItem onClick={() => viewDeal(row.original.id)}>
            <Eye className="mr-2 w-4 h-4" /> View Details
          </DropdownMenuItem>
          <DropdownMenuItem onClick={() => downloadReport(row.original.id)}>
            <Download className="mr-2 w-4 h-4" /> Download Report
          </DropdownMenuItem>
          <DropdownMenuItem onClick={() => shareWithPartner(row.original.id)}>
            <Share className="mr-2 w-4 h-4" /> Share with Partner
          </DropdownMenuItem>
          <DropdownMenuSeparator />
          <DropdownMenuItem onClick={() => archiveDeal(row.original.id)}>
            <Archive className="mr-2 w-4 h-4" /> Archive
          </DropdownMenuItem>
        </DropdownMenuContent>
      </DropdownMenu>
    )
  }
]
```

```
  return (
    <DataTable
      columns={columns}
      data={deals}
      sortable
      filterable
      pagination
    />
  )
}
```

---

5. Advanced Components

5.1 Score Visualization Components

```
 // components/shared/Charts/ScoreRing.tsx
import { PieChart, Pie, Cell } from 'recharts'

export function ScoreRing({ score, size = 'md' }) {
  const sizes = {
    sm: 120,
    md: 180,
    lg: 240,
    xl: 300
  }

  const diameter = sizes[size]
  const data = [
    { value: score },
    { value: 100 - score }
  ]

  const color = getScoreColor(score)

  return (
    <div className="relative inline-block">
      <PieChart width={diameter} height={diameter}>
        <Pie
          data={data}
          cx={diameter / 2}
          cy={diameter / 2}
          innerRadius={diameter * 0.35}
          outerRadius={diameter * 0.45}
          startAngle={90}
          endAngle={-270}
          dataKey="value"
```

```
        >
          <Cell fill={color} />
          <Cell fill="#E5E7EB" />
        </Pie>
      </PieChart>
      <div className="absolute inset-0 flex items-center justify-center">
        <div className="text-center">
          <div className="text-4xl font-bold">{score}</div>
          <div className="text-sm text-neutral-500">/ 100</div>
        </div>
      </div>
    </div>
  )
}

// components/shared/Charts/PercentileChart.tsx
export function PercentileChart({ percentile }) {
  return (
    <div className="relative h-32 bg-neutral-100 rounded-lg overflow-hidden">
      {/* Background gradient */}
      <div className="absolute inset-0 bg-gradient-to-r from-red-200 via-yellow-200 via-blue-200 to-green-200" />

      {/* Marker */}
      <div
        className="absolute top-0 bottom-0 w-1 bg-neutral-900"
        style={{ left: ${percentile}%`}}
      >
        <div className="absolute -top-2 left-1/2 -translate-x-1/2 bg-neutral-900 text-white text-xs px-2 py-1 rounded whites
          You ({percentile}%)
        </div>
      </div>

      {/* Labels */}
      <div className="absolute bottom-2 left-2 text-xs text-neutral-600">
        Bottom
      </div>
      <div className="absolute bottom-2 right-2 text-xs text-neutral-600">
        Top
      </div>
    </div>
  )
}

// components/shared/Charts/ReadinessChart.tsx
import { LineChart, Line, XAxis, YAxis, Tooltip, CartesianGrid } from 'recharts'

export function ReadinessChart({ data }) {
  return (
    <LineChart width={800} height={300} data={data}>
      <CartesianGrid strokeDasharray="3 3" />
```

```
      <XAxis dataKey="date" />
      <YAxis domain={[0, 100]} />
      <Tooltip />
      <Line
        type="monotone"
        dataKey="overall_score"
        stroke="#2563EB"
        strokeWidth={3}
        dot={{ r: 6 }}
      />
      <Line
        type="monotone"
        dataKey="pitch_quality"
        stroke="#7C3AED"
        strokeWidth={2}
      />
      <Line
        type="monotone"
        dataKey="traction"
        stroke="#10B981"
        strokeWidth={2}
      />
    </LineChart>
  )
}
```

5.2 File Upload Component

```
// components/shared/FileUpload.tsx
import { useDropzone } from 'react-dropzone'
import { Upload, File, X } from 'lucide-react'

export function FileUpload({ accept, maxSize, onUpload }) {
  const [file, setFile] = useState(null)
  const [uploading, setUploading] = useState(false)

  const { getRootProps, getInputProps, isDragActive } = useDropzone({
    accept,
    maxSize,
    multiple: false,
    onDrop: async (acceptedFiles) => {
      const file = acceptedFiles[0]
      setFile(file)
      setUploading(true)

      try {
        await onUpload(file)
      } catch (error) {
```

```
      toast.error('Upload failed')
    } finally {
      setUploading(false)
    }
  }
})

return (
  <div>
    {!file ? (
      <div
        {...getRootProps()}
        className={cn(
          'border-2 border-dashed rounded-lg p-12 text-center cursor-pointer transition-colors',
          isDragActive ? 'border-blue-500 bg-blue-50' : 'border-neutral-300'
        )}
      >
        <input {...getInputProps()} />
        <Upload className="w-16 h-16 mx-auto mb-4 text-neutral-400" />
        {isDragActive ? (
          <p className="text-blue-600">Drop your file here</p>
        ) : (
          <div>
            <p className="text-lg font-medium">
              Drag & drop your pitch deck here
            </p>
            <p className="text-sm text-neutral-500 mt-2">
              or click to browse
            </p>
            <p className="text-xs text-neutral-400 mt-4">
              Supports PDF, PPT, PPTX (max {maxSize / 1024 / 1024}MB)
            </p>
          </div>
        )}
      </div>
    ) : (
      <Card>
        <div className="flex items-center justify-between p-4">
          <div className="flex items-center gap-3">
            <File className="w-8 h-8 text-blue-500" />
            <div>
              <p className="font-medium">{file.name}</p>
              <p className="text-sm text-neutral-500">
                {(file.size / 1024 / 1024).toFixed(2)} MB
              </p>
            </div>
          </div>

          {uploading ? (
            <Loader2 className="w-6 h-6 animate-spin" />
```

```
          ) : (
            <Button
              variant="ghost"
              size="sm"
              onClick={() => setFile(null)}
            >
              <X className="w-4 h-4" />
            </Button>
          )}
        </div>

        {uploading && (
          <Progress value={uploadProgress} className="h-1" />
        )}
      </Card>
    )}
  </div>
 )
}
```

---

6. State Management

6.1 Zustand Stores

```
 // lib/stores/authStore.ts
import { create } from 'zustand'
import { persist } from 'zustand/middleware'

interface AuthState {
  user: User | null
  org: Organization | null
  token: string | null
  login: (email: string, password: string) => Promise<void>
  logout: () => void
  updateUser: (user: Partial<User>) => void
}

export const useAuthStore = create<AuthState>()(
  persist(
    (set, get) => ({
      user: null,
      org: null,
      token: null,

      login: async (email, password) => {
        const response = await api.auth.login(email, password)
        set({
```

```
          user: response.user,
          org: response.org,
          token: response.token
        })
      },

      logout: () => {
        set({ user: null, org: null, token: null })
      },

      updateUser: (updates) => {
        set((state) => ({
          user: { ...state.user, ...updates }
        }))
      }
    }),
    {
      name: 'auth-storage'
    }
  )
)

// lib/stores/startupStore.ts
interface StartupState {
  currentStartup: Startup | null
  startups: Startup[]
  setCurrentStartup: (id: string) => void
  updateStartup: (updates: Partial<Startup>) => void
}

export const useStartupStore = create<StartupState>((set, get) => ({
  currentStartup: null,
  startups: [],

  setCurrentStartup: (id) => {
    const startup = get().startups.find(s => s.id === id)
    set({ currentStartup: startup })
  },

  updateStartup: (updates) => {
    set((state) => ({
      currentStartup: { ...state.currentStartup, ...updates }
    }))
  }
}))
```

6.2 React Query Configuration

```
// lib/queryClient.ts
import { QueryClient } from '@tanstack/react-query'

export const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      staleTime: 5 * 60 * 1000, // 5 minutes
      cacheTime: 10 * 60 * 1000, // 10 minutes
      retry: 2,
      refetchOnWindowFocus: false
    },
    mutations: {
      onError: (error) => {
        toast.error(error.message)
      }
    }
  }
})

// Custom hooks
export function useStartupAnalysis(startupId: string) {
  return useQuery({
    queryKey: ['analysis', startupId],
    queryFn: () => api.getAnalysis(startupId),
    enabled: !!startupId
  })
}

export function useAnalyzePitchDeck() {
  const queryClient = useQueryClient()

  return useMutation({
    mutationFn: (data: AnalysisRequest) => api.analyzePitchDeck(data),
    onSuccess: (data) => {
      queryClient.invalidateQueries(['analysis', data.startup_id])
      queryClient.invalidateQueries(['readiness', data.startup_id])
      toast.success('Analysis complete!')
    }
  })
}
```

---

7. API Client Architecture

```
// lib/api/client.ts
import axios from 'axios'
import { useAuthStore } from '@/lib/stores/authStore'
```

```
const apiClient = axios.create({
  baseURL: process.env.NEXT_PUBLIC_API_URL || 'http://localhost:8000',
  timeout: 30000
})

// Request interceptor
apiClient.interceptors.request.use((config) => {
  const token = useAuthStore.getState().token
  if (token) {
    config.headers.Authorization = Bearer ${token}`
  }
  return config
})

// Response interceptor
apiClient.interceptors.response.use(
  (response) => response.data,
  (error) => {
    if (error.response?.status === 401) {
      useAuthStore.getState().logout()
      window.location.href = '/login'
    }
    throw error
  }
)

// lib/api/services/pitchDeckService.ts
export const pitchDeckService = {
  analyze: async (file: File, metadata: PitchMetadata) => {
    const formData = new FormData()
    formData.append('file', file)
    Object.entries(metadata).forEach(([key, value]) => {
      formData.append(key, value)
    })

    return apiClient.post('/api/v1/pitch-deck/analyze', formData, {
      headers: { 'Content-Type': 'multipart/form-data' }
    })
  },

  getAnalysis: (analysisId: string) => {
    return apiClient.get(`/api/v1/pitch-deck/analysis/${analysisId}`)
  },

  listAnalyses: (startupId: string) => {
    return apiClient.get(`/api/v1/pitch-deck/analyses`, {
      params: { startup_id: startupId }
    })
  }
}
```

```
// lib/api/index.ts
export const api = {
  auth: authService,
  pitchDeck: pitchDeckService,
  readiness: readinessService,
  valuation: valuationService,
  investors: investorService,
  communities: communityService,
  crm: crmService,
  analytics: analyticsService
}
```

---

8. Responsive Design Strategy

```
 // Breakpoints (Tailwind)
const breakpoints = {
  sm: '640px',   // Mobile landscape
  md: '768px',   // Tablet
  lg: '1024px',  // Desktop
  xl: '1280px',  // Large desktop
  '2xl': '1536px' // Extra large
}

// Mobile-first approach
<div className="
  px-4 py-6           // Mobile
  md:px-8 md:py-8     // Tablet
  lg:px-12 lg:py-12   // Desktop
">
  {/* Content */}
</div>

// Responsive grid
<div className="
  grid
  grid-cols-1        // Mobile: 1 column
  md:grid-cols-2     // Tablet: 2 columns
  lg:grid-cols-3     // Desktop: 3 columns
  gap-4
">
  {/* Cards */}
</div>

// Conditional rendering
{isMobile ? (
  <MobileNavigation />
```

```
) : (
  <DesktopSidebar />
)}
```

---

9. Performance Optimization

9.1 Code Splitting

```
// Dynamic imports for large components
const PitchAnalyzer = dynamic(
  () => import('@/components/founder/PitchAnalyzer'),
  { loading: () => <Skeleton /> }
)

const ValuationCalculator = dynamic(
  () => import('@/components/founder/ValuationCalculator'),
  { ssr: false }
)
```

9.2 Image Optimization

```
import Image from 'next/image'

<Image
  src="/startup-logo.png"
  alt="Startup Logo"
  width={100}
  height={100}
  loading="lazy"
  placeholder="blur"
/>
```

9.3 Memoization

```
const expensiveComponent = useMemo(() => (
  <ComplexChart data={chartData} />
), [chartData])

const sortedDeals = useMemo(() => {
  return deals.sort((a, b) => b.score - a.score)
}, [deals])
```

---

10. Accessibility (a11y)

```
// Keyboard navigation
<Button
  onClick={handleClick}
  onKeyDown={(e) => {
    if (e.key === 'Enter' || e.key === ' ') {
      handleClick()
    }
  }}
  aria-label="Analyze pitch deck"
>
  Analyze
</Button>

// ARIA attributes
<div
  role="progressbar"
  aria-valuenow={score}
  aria-valuemin={0}
  aria-valuemax={100}
  aria-label="Overall score"
>
  <Progress value={score} />
</div>

// Focus management
const inputRef = useRef<HTMLInputElement>(null)

useEffect(() => {
  inputRef.current?.focus()
}, [])

// Color contrast
// Ensure all text meets WCAG AA standards (4.5:1 contrast ratio)
```

---

11. Testing Strategy

11.1 Component Tests

```
// __tests__/components/ScoreCard.test.tsx
import { render, screen } from '@testing-library/react'
import { ScoreCard } from '@/components/shared/ScoreCard'

describe('ScoreCard', () => {
  it('renders score correctly', () => {
    render(<ScoreCard title="Idea" score={75} />)
    expect(screen.getByText('75')).toBeInTheDocument()
  })
```

```
  it('shows correct color for score', () => {
    const { container } = render(<ScoreCard title="Team" score={85} />)
    expect(container.querySelector('.bg-green-500')).toBeInTheDocument()
  })
})
```

11.2 Integration Tests

```
 // __tests__/flows/pitch-analysis.test.tsx
import { render, waitFor, fireEvent } from '@testing-library/react'
import PitchAnalyzer from '@/app/founder/pitch-analyzer/page'

describe('Pitch Analysis Flow', () => {
  it('completes full analysis', async () => {
    const { getByText, getByLabelText } = render(<PitchAnalyzer />)

    // Upload file
    const file = new File(['content'], 'pitch.pdf', { type: 'application/pdf' })
    const input = getByLabelText('Upload pitch deck')
    fireEvent.change(input, { target: { files: [file] } })

    // Fill metadata
    fireEvent.change(getByLabelText('Sector'), { target: { value: 'AI/ML' } })

    // Submit
    fireEvent.click(getByText('Analyze Now'))

    // Wait for results
    await waitFor(() => {
      expect(getByText(/Overall Score/i)).toBeInTheDocument()
    })
  })
})
```

11.3 E2E Tests (Playwright)

```
 // e2e/founder-journey.spec.ts
import { test, expect } from '@playwright/test'

test('founder can analyze pitch and check readiness', async ({ page }) => {
  await page.goto('/login')

  // Login
  await page.fill('[name=email]', 'founder@startup.com')
  await page.fill('[name=password]', 'password')
  await page.click('button[type=submit]')
```

```
  // Navigate to pitch analyzer
  await page.click('text=Analyze Pitch')

  // Upload deck
  await page.setInputFiles('input[type=file]', './fixtures/pitch.pdf')
  await page.selectOption('[name=sector]', 'AI/ML')
  await page.click('text=Analyze Now')

  // Wait for analysis
  await page.waitForSelector('text=Overall Score', { timeout: 180000 })

  // Verify score is displayed
  const score = await page.textContent('[data-testid=overall-score]')
  expect(Number(score)).toBeGreaterThan(0)

  // Check readiness
  await page.click('text=Check Investment Readiness')
  await page.waitForSelector('text=Readiness Score')
})
```

---

12. Deployment & DevOps

12.1 Environment Configuration

```
 // .env.local
NEXT_PUBLIC_API_URL=http://localhost:8000
NEXT_PUBLIC_ENV=development

// .env.production
NEXT_PUBLIC_API_URL=https://api.platform.com
NEXT_PUBLIC_ENV=production
NEXT_PUBLIC_SENTRY_DSN=...
```

12.2 Build Configuration

```
 // next.config.js
module.exports = {
  reactStrictMode: true,
  swcMinify: true,
  images: {
    domains: ['s3.amazonaws.com']
  },
  env: {
    API_URL: process.env.NEXT_PUBLIC_API_URL
  },
```

```
  webpack: (config) => {
    config.module.rules.push({
      test: /\.md$/,
      use: 'raw-loader'
    })
    return config
  }
}
```

12.3 CI/CD Pipeline

```
# .github/workflows/deploy-frontend.yml
name: Deploy Frontend

on:
  push:
    branches: [main]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2

      - name: Setup Node
        uses: actions/setup-node@v2
        with:
          node-version: '18'

      - name: Install dependencies
        run: npm ci

      - name: Run tests
        run: npm test

      - name: Build
        run: npm run build

      - name: Deploy to Vercel
        run: vercel deploy --prod --token=${{ secrets.VERCEL_TOKEN }}
```

---

13. Phase-wise Implementation Roadmap

Phase 1: Foundation (Weeks 1-2)

- [ ] Set up Next.js project with TypeScript

- [ ] Install and configure shadcn/ui
- [ ] Set up authentication flow
- [ ] Build layout components (Nav, Sidebar)
- [ ] Create design system tokens
- [ ] Set up API client

Phase 2: Founder Core Features (Weeks 3-5)

- [ ] Pitch Deck Analyzer UI
- [ ] Investment Readiness Dashboard
- [ ] Valuation Benchmarking interface
- [ ] Founder profile & settings
- [ ] File upload handling
- [ ] Results visualization

Phase 3: VC Dashboard (Weeks 6-8)

- [ ] Deal flow table
- [ ] Deal detail views
- [ ] Analytics dashboard
- [ ] Startup comparison tools
- [ ] Reporting & exports
- [ ] Team collaboration features

Phase 4: Advanced Features (Weeks 9-11)

- [ ] Investor matching interface
- [ ] Fundraising CRM
- [ ] Community platform
- [ ] Real-time notifications
- [ ] Advanced search & filters
- [ ] Mobile optimization

Phase 5: Polish & Launch (Weeks 12-14)

- [ ] Performance optimization
- [ ] Accessibility audit
- [ ] Browser testing
- [ ] User acceptance testing
- [ ] Documentation
- [ ] Production deployment

---

14. Success Metrics

Technical KPIs

First Contentful Paint < 1.5s
Time to Interactive < 3s
Lighthouse score > 90
Zero critical a11y violations
Test coverage > 80%

User Experience KPIs

Deck analysis completion rate > 85%
Time to first insight < 4 minutes
Dashboard load time < 2s
Mobile responsiveness score > 95

---

15. Risk Mitigation

Risk	Impact	Mitigation
Large file uploads slow	High	Chunked uploads, compression, progress indicators
Complex data visualization performance	Medium	Canvas rendering, virtualization, lazy loading
Browser compatibility issues	Medium	Polyfills, automated testing, graceful degradation
API latency	High	Optimistic updates, caching, loading states
Design inconsistency	Low	Design system, component library, code reviews

---

Conclusion

This frontend plan delivers a **dual-experience platform** optimized for both VCs and founders with:

Speed - Sub-3-second load times, optimistic updates
Clarity - Data-dense yet scannable interfaces
Professionalism - Enterprise-grade design system

Accessibility - WCAG AA compliant
Scalability - Modular architecture, code splitting

Next Steps:

Review design mockups
Set up development environment
Build component library
Begin Phase 1 implementation
Weekly design reviews with stakeholders